Action Return Types in ASP.NET Core Web API

What are controller action return types?

In ASP.NET Core Web API, a **Controller Action Return Type** defines what data and how data is sent back to the client after the controller finishes processing a request.

Think of it as the **promise of response**. The action method executes logic, fetches or process data, and then returns something to the client. That "something" could be:

- A **Simple Value** like an integer representing a count
- A **Complex Object** like 'Cloth' or 'Order'
- Or an **HTTP-aware response** using helpers like 'Ok()', 'NotFound()', or 'BadRequest()'

The ASP.NET Core runtime serializes this return value into JSON and sends it in the HTTP response body.

Why Return Type Matters:

- Determines HTTP Status Code (200, 404, 400, 201, etc.)
- Determines Response Body Content (data or error message)
- Determines Client's Behavior (clients depend on HTTP response type)

Categories of Return Types

In ASP.NET Core Web API, runtime types fall into three main categories:

- ➔ Specific Types (e.g., ‘string’, ‘int’, ‘Cloth’)
- ➔ IActionResult/ActionResult
- ➔ ActionResult<T>

Specific Types (e.g., ‘string’, ‘int’, ‘Cloth’)

These are direct return types. ASP.NET Core automatically assumes a successful request and sends back HTTP 200 OK.

- If a reference type returns null, the framework sends 204 No Content.
- This approach is simple but lacks flexibility for real-world error handling.
- It has no control over HTTP responses.
- Strong type safety
- Support Asynchronous: **Task<T>**
- Best for Learning or simple APIs

IActionResult/ActionResult

These return types give us complete control over the HTTP response. We can manually decide:

- This status code (e.g., 200, 404, 400),
- The response message,
- And even attach headers.
- Comparatively weak type safety
- Support Asynchronous: **Task<IActionResult>**

We can return various predefined results, such as:

- Ok(object) → 200 OK

- NotFound(string) → 404 Bad Request
- CreatedAtAction() → 201 Created

This perfect when we want to return different outcomes (success, error, not found, etc.) within the same method. It is used heavily in real-world REST APIs.

ActionResult<T>

This is a **Generic Return Type** that merges the benefits of both previous categories. It is introduced in ASP.NET Core 2.1, which combines:

- **Strong Typing** (Swagger knows exactly what T is)
- **Flexible HTTP Control** (we can still return Ok(), NotFound(), BadRequest())

- It has the strong **Type Safety**.
- Supports Asynchronous: **Task<ActionResult<T>>**

Hence, it's considered the **best practice for modern APIs**.

Async Methods in ASP.NET Core Web API

What are Async Methods?

Async methods allow non-blocking operations. When a method is marked as **async** and returns **Task<T>**, the thread is freed up to handle other requests while waiting for I/O operations (database calls, external API requests, file operations).

Why Use Async Methods?

- Improves scalability (frees threads during I/O wait time)

- Better resource utilization
- Prevents thread pool starvation under load
- Standard pattern for modern .NET applications

Async Return types Table:

Synchronous Return Type	Async Return Type	Method Pattern
int	Task<int>	public async Task<int> GetCountAsync()
Cloth	Task<Cloth?>	public async Task<Cloth?> GetByIdAsync(int id)
List<Cloth>	Task<List<Cloth>>	public async Task<List<Cloth>> GetAllAsync()
IActionResult	Task<IActionResult>	public async Task<IActionResult> GetDetailsAsync(int id)
ActionResult<T>	Task<ActionResult<T>>	public async Task<ActionResult<Cloth>> GetByIdAsync(int id)

Syntax Pattern:

```
// Synchronous
public Cloth? GetClothById(int id) { ... }

// Asynchronous (same return type + Task wrapper + async keyword + await)
public async Task<Cloth?> GetClothByIdAsync(int id)
{
    return await _context.Cloths.FindAsync(id);
}
```

Now we are going to set up a project ‘ClothsApi’ to represent the above concepts,

Project Setup

<u>Create the project:</u>

Command: dotnet new webapi -n ClothsApi -f net8.0

<u>Navigate to the project:</u>

Command: cd ClothsApi

Install Packages

Before installing the packages, we should make sure that we are in the project folder ‘ClothsApi’ in terminal. If not, then navigate to the exact project folder in terminal.

Command 1: dotnet add package Microsoft.EntityFrameworkCore --version 8.0.0

Command 2: dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0

Command 3: dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.0

Command 4: dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.0

To check, whether the packages are successfully set up or not, we have to look over .csproj file,

ClothsApi.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="8.0.24" />
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>

</Project>
```

Step 1: Create Cloth Model

Path: ClothsApi/Models/Cloth.cs

Cloth.cs

Models > Cloth.cs > ...

```
namespace ClothsApi.Models
{
    // Represents a Cloth/Cloathing item in our online shop
    0 references
    public class Cloth
    {
        // Unique identifier
        0 references
        public int Id { get; set; }

        // Name, e.g., 'Cotton T-Shirt'
        0 references
        public string Name { get; set; } = string.Empty;

        // Category, e.g., 'Men', 'Women', 'Kids'
        0 references
        public string Category { get; set; } = string.Empty;

        // Type, e.g., 'Shirt', 'Pant', 'Jacket'
        0 references
        public string Type { get; set; } = string.Empty;

        // Size, e.g., 'S', 'M','L', 'XL'
        0 references
        public string Size { get; set; } = string.Empty;

        // Color, e.g., 'Red', 'Blue', 'Black'
        0 references
        public string Color { get; set; } = string.Empty;

        // Price in decimal
        0 references
        public decimal Price { get; set; }
        // Avaibablity flag
        0 references
        public bool InStock { get; set; }
    }
}
```

Step 2: Create Cloth Service (Async Methods with In-Memory Data)

Path: ClothsApi/Services/ClothService.cs

ClothService.cs

Services > ClothService.cs > ...

```
using ClothsApi.Models;

namespace ClothsApi.Services
{
    0 references
    public static class ClothService
    {
        // In-memory product list simulating a database table
        5 references
        private static readonly List<Cloth> _cloths = new()
        {
            new Cloth {
                Id = 1,
                Name = "Cotton T-Shirt",
                Category = "Men",
                Type = "Shirt",
                Size = "M",
                Color = "Blue",
                Price = 599,
                InStock = true
            },
            new Cloth {
                Id = 2,
                Name = "Denim Jacket",
                Category = "Women",
                Type = "Jacket",
                Size = "L",
                Color = "Black",
                Price = 2499,
                InStock = true
            },
            new Cloth {
                Id = 3,
                Name = "Formal Shirt",
                Category = "Men",
                Type = "Shirt",
                Size = "XL",
                Color = "White",
                Price = 1299,
                InStock = false
            },
            new Cloth {
                Id = 4,
                Name = "Sports Shoes",
                Category = "Men",
                Type = "Shoes",
                Size = "9",
                Color = "Grey",
                Price = 3499,
                InStock = true
            },
            new Cloth {
                Id = 5,
                Name = "Silk Saree",
                Category = "Women",
                Type = "Saree",
                Size = "Free",
                Color = "Red",
                Price = 4999,
                InStock = true
            },
            new Cloth {
                Id = 6,
                Name = "Kids Hoodie",
                Category = "Kids",
                Type = "Hoodie",
                Size = "S",
                Color = "Yellow",
                Price = 899,
                InStock = true
            },
            new Cloth {
                Id = 7,
                Name = "Leather Belt",
                Category = "Accessories",
                Type = "Belt",
                Size = "32",
                Color = "Brown",
                Price = 399,
                InStock = false
            }
        };

        // ============================================================
        // ALL SERVICE METHODS ARE ASYNC (Simulate database I/O)
        // Returns Task<T> wrapper for async operations
        // ============================================================

        // Async: Returns total number of cloth items (primitive type)
        0 references
        public static async Task<int> GetClothCountAsync()
        {
            await Task.Delay(100); // Simulate async DB call
            return _cloths.Count;
        }

        // Async: Returns all cloth item (collection of complex type)
        0 references
        public static async Task<List<Cloth>> GetAllClothsAsync()
        {
            await Task.Delay(200);
            return _cloths;
        }
```

```
100
101         // Async: Returns a single cloth by ID (complex type or null)
            0 references
102         public static async Task<Cloth?> GetClothByIdAsync(int id)
103         {
104             await Task.Delay(100);
105             return _cloths.FirstOrDefault(c => c.Id == id);
106         }
107
108         // Async: Returns only cloth names (collection of primitive types)
            0 references
109         public static async Task<List<string>> GetAllClothNamesAsync()
110         {
111             await Task.Delay(150);
112             return _cloths.Select(c => c.Name).ToList();
113         }
```

```
114
115         // Async: Returns cloths by catrgory (filtered collection)
            0 references
116         public static async Task<List<Cloth>> GetClothsByCategoryAsync(string category)
117         {
118             await Task.Delay(100);
119             return _cloths.Where(
120                 c => c.Category.Equals(
121                     category, StringComparison.OrdinalIgnoreCase
122                 )
123             ).ToList();
124         }
125     }
126 }
```

Step 3: Program.cs (Minimal)

Path: ClothsApi/Program.cs

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();

```

Step 4: Category 1 – Returning Specific Types (Async)

Path: ClothsApi/Controllers/ClothController.cs

ClothController.cs

Controllers > ClothController.cs > ...

```
using ClothsApi.Models;
using ClothsApi.Services;
using Microsoft.AspNetCore.Mvc;

namespace ClothsApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    0 references
    public class ClothController : ControllerBase
    {
        // ============================================================
        // CATEGORY 1: SPECIFIC TYPES (Primitive and Complex)
        // ASYNC: Task<int>, Task<Cloth?>, Task<List<Cloth>>, Task<List<string>>
        // LIMITATION: Always returns 200 OK - No control over status codes
        // ============================================================

        // GET /api/cloth/GetClothCount
        // Returns: Task<int> - 200 OK with integer value (number of cloths)
        [HttpGet("GetClothCount")]
        0 references
        public async Task<int> GetClothCount()
        {
            return await ClothService.GetClothCountAsync();
        }

```

```
        // GET /api/cloth/GetClothById/3
        // Returns: Task<Cloth?> - 200 OK with Cloth object OR 204 No Content (if null)
        // ISSUE: Cannot return 404 Not Found for missing items
        [HttpGet("GetClothById/{id}")]
        0 references
        public async Task<Cloth?> GetClothById(int id)
        {
            return await ClothService.GetClothByIdAsync(id);
        }

        // GET /api/cloth/GetAllCloths
        // Returns: Task<List<Cloth>> - 200 OK with list of cloths
        [HttpGet("GetAllCloths")]
        0 references
        public async Task<List<Cloth>> GetAllCloths()
        {
            return await ClothService.GetAllClothsAsync();
        }

```

```
        // GET /api/cloth/GetAllClothNames
        // Returns: Task<List<string>> - 200 OK with list of names only
        [HttpGet("GetAllClothNames")]
        0 references
        public async Task<List<string>> GetAllClothNames()
        {
            return await ClothService.GetAllClothNamesAsync();
        }
    }
}
```

Async Pattern for Specific Types:

Synchronous	**Asynchronous**
public int GetCount()	public async Task<int> GetCountAsync()
public Cloth? GetById(int id)	public async Task<Cloth?> GetByIdAsync(int id)
return data;	return await service.GetDataAsync();

Limitations of Specific Types:

- No HTTP Status Code Control (cannot return 404, 400, 201)
- No Standardized Error Format
- Cannot return custom error messages
- Not suitable for Production APIs

Step 5: Category 2 – Returning IActionResult (Async)

Add to ClothsApi/Controllers/ClothController.cs:

```
50      // ==============================================================
51      // CATEGORY 2: IActionResult - FULL HTTP CONTROL
52      // ASYNC: Task<IActionResult>
53      // BEST FOR: Real-world APIs with proper error handling
54      // ==============================================================
55
56      // GET /api/cloth/GetClothDetails/5
57      // Returns: Task<IActionResult> - 200 OK, OR 400 Bad Request, OR
58      // 404 NOT Found
59      [HttpGet("GetClothDetails/{id}")]
        0 references
60      public async Task<IActionResult> GetClothDetails(int id)
61      {
62          // Bad Request - Invalid input
63          if (id <= 0)
64              return BadRequest("Invalid cloth ID. Must be greater than zero.");
65
66          var cloth = await ClothService.GetClothByIdAsync(id);
67
68          // Not Found - Resource doesn't exist
69          if (cloth == null)
70              return NotFound($"Cloth with ID = {id} was not found.");
71
72          // Success - Return data with 200 OK
73          return Ok(cloth);
74      }
75
```

```
76      // GET /api/cloth/GetClothsByCategory/men
77      // Returns: Task<IActionResult> - 200 OK with filtered list, OR 404 if
78      // no items found
79      [HttpGet("GetClothsByCategory/{category}")]
        0 references
80      public async Task<IActionResult> GetClothsByCategory(string category)
81      {
82          if (string.IsNullOrWhiteSpace(category))
83              return BadRequest("Category cannot be empty.");
84
85
86          var cloths = await ClothService.GetClothsByCategoryAsync(category);
87
88          if (cloths == null || cloths.Count == 0)
89              return NotFound($"No cloths found in category: {category}");
90
91          return Ok(cloths);
92      }
93
94      // GET /api/cloth/GetAllClothsWithStatus
95      // Demonstrates: NoContent (204) when list is empty
96      [HttpGet("GetAllClothsWithStatus")]
        0 references
97      public async Task<IActionResult> GetAllClothsWithStatus()
98      {
99          var cloths = await ClothService.GetAllClothsAsync();
100
101         if (cloths == null || cloths.Count == 0)
102             return NoContent();     // 204 No Content
103
104         return Ok(cloths);
105     }
```

Async Pattern for IActionResult:

Synchronous	Asynchronous
public IActionResult GetDetails(int id)	public async Task<IActionResult> GetDetailsAsync(int id)
var data = services.GetById(id);	var data = await services.GetByIdAsync(id);
return Ok(data);	return Ok(data);

IActionResult Helper Methods Reference:

Method	HTTP Status Code	Use Case
Ok(data)	200 OK	Successful response with data
NotFound(message)	404 Not Found	Resource doesn't exist
BadRequest(message)	400 Bad Request	Invalid input from client
NoContent()	204 No Content	Success but no data to return

Limitations of IActionResult:

- No compile-time type safety
- Swagger shows "object" instead of actual schema
- Can accidentally return wrong data type

Step 6: Category 3 – Returning ActionResult<T> (Best Practice - Async)

Add to ClothsApi/Controllers/ClothController.cs:

```
107         // ================================================================
108         // CATEGORY 3: ActionResult<T> - BEST PRACTICE
109         // ASYNC: Task<ActionResult<T>>
110         // Combines Strong Typing + Full HTTP Control
111         // Swagger shows exact response schema
112         // ================================================================
113
114         // GET /api/cloth/GetClothCountTyped
115         // Returns: Task<ActionResult<int>> - Strongly typed integer response
116         [HttpGet("GetClothCountTyped")]
            0 references
117         public async Task<ActionResult<int>> GetClothCountTyped()
118         {
119             int count = await ClothService.GetClothCountAsync();
120             return Ok(count);       // 200 OK with integer
121         }
122
123         // GET /api/cloth/GetClothByIdTyped/3
124         // Returns: Task<ActionResult<Cloth>> - Strongly typed Cloth object
125         [HttpGet("GetClothByIdTyped/{id}")]
            0 references
126         public async Task<ActionResult<Cloth>> GetClothByIdTyped(int id)
127         {
128             if (id <= 0)
129                 return BadRequest("Invalid cloth ID. Must be greater than zero.");
130
131             var cloth = await ClothService.GetClothByIdAsync(id);
132
133             if (cloth == null)
134                 return NotFound($"Cloth with ID = {id} was not found.");
135
136             return Ok(cloth);
137         }
```

```
138
139         // GET /api/cloth/GetAllClothsTyped
140         // Returns: Task<ActionResult<List<Cloth>>> - Strongly typed list
141         [HttpGet("GetAllClothsTyped")]
            0 references
142         public async Task<ActionResult<List<Cloth>>> GetAllClothsTyped()
143         {
144             var cloths = await ClothService.GetAllClothsAsync();
145
146             if (cloths == null || cloths.Count == 0)
147                 return NoContent();
148
149             return Ok(cloths);
150         }
151
152         // GET /api/cloth/GetClothNamesTyped
153         // Returns: Task<ActionResult<List<string>>> - Strongly typed string list
154         [HttpGet("GetClothNamesTyped")]
            0 references
155         public async Task<ActionResult<List<string>>> GetClothNamesTyped()
156         {
157             var names = await ClothService.GetAllClothNamesAsync();
158
159             if (names == null || names.Count == 0)
160                 return NotFound("No cloth names found.");
161
162             return Ok(names);
163         }
```

```
        // GET /api/cloth/GetByCategoryTyped/men
        // Returns: Task<ActionResult<list<Cloth>>> - Filtered by category
        [HttpGet("GetByCategoryTyped/{category}")]
        0 references
        public async Task<ActionResult<List<Cloth>>> GetByCategoryTyped(string category)
        {
            if(string.IsNullOrWhiteSpace(category))
                return BadRequest("Category cannot be empty.");

            var cloths = await ClothService.GetClothsByCategoryAsync(category);

            if (cloths == null || cloths.Count == 0)
                return NotFound($"No cloths found in category: {category}");

            return Ok(cloths);
        }
    }
}
```

Async Pattern for ActionResult<T>:

Synchronous	**Asynchronous**
public ActionResult<Cloth> GetById(int id)	public async Task<ActionResult<Cloth>> GetByIdAsync(int id)
var data = services.GetById(id);	var data = await service.GetByIdAsync(id);
return Ok(data);	return Ok(data);

Why ActionResult<T> is Best Practice?

We prefer ActionResult<T> because it offers the perfect balance between type safety and flexibility in our API controllers. By using it, we ensure that documentation tools like Swagger automatically recognize our data models without extra configuration. We can still return various HTTP status codes, such as NotFound() or BadRequest(), whenever a request doesn't go as planned.

This approach simplifies our development process by allowing us to return data objects directly, thanks to implicit casting. It also makes our unit tests much cleaner since we can access the returned value without complex type checking. Ultimately, we adopt this best practice to keep our code readable, maintainable, and well-documented for other developers.

Test our API – All Endpoints with Expected Responses

Let's build and run the application. And here we will use the command line or cli. We have to navigate to the exact project folder (ClothsApi) and run those following commands,

Build the application,

Command: *dotnet build*

```
PS E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi> dotnet build
  Determining projects to restore...
  All projects are up-to-date for restore.
  ClothsApi -> E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi\bin\Debug\
net8.0\ClothsApi.dll
  1

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:01.98
```

To run the application,

Command: *dotnet run* (we usually get the URL from here to see application response).

We can run the project and can directly navigate to application response or Swagger UI.

Use can run the command,

Command: *dotnet watch run*

```
PS E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi> dotnet watch run
dotnet watch 🔥 Hot reload enabled. For a list of supported edits, see https://aka.ms/dot
net/hot-reload.
   💡 Press "Ctrl + R" to restart.
dotnet watch 🔧 Building...
  Determining projects to restore...
  All projects are up-to-date for restore.
  ClothsApi -> E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi\bin\Debug
  \net8.0\ClothsApi.dll
dotnet watch 🚀 Started
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5146
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: E:\PracticeProjects\DotnetApi\ReturnTypes&StatusCodes\ClothsApi
```

localhost:5146/swagger/index.html

Import favorites | Active nowCSE Ru... | YouTube | Other favorit

ClothsApi 1.0 OAS 3.0

http://localhost:5146/swagger/v1/swagger.json

Cloth

- GET /api/Cloth/GetClothCount
- GET /api/Cloth/GetClothById/{id}
- GET /api/Cloth/GetAllCloths
- GET /api/Cloth/GetAllClothNames
- GET /api/Cloth/GetClothDetails/{id}
- GET /api/Cloth/GetClothsByCategory/{category}
- GET /api/Cloth/GetAllClothsWithStatus
- GET /api/Cloth/GetClothCountTyped
- GET /api/Cloth/GetClothByIdTyped/{id}
- GET /api/Cloth/GetAllClothsTyped
- GET /api/Cloth/GetClothNamesTyped
- GET /api/Cloth/GetByCategoryTyped/{category}

Schemas

Cloth ›

CATEGORY 1 TESTS: Specific Types (Async – Task<T>)

TEST 1.1: Get Count (Task<int>)

Request: GET http://localhost:5146/api/Cloth/GetClothCount

Response (200 OK):

Request URL

```
http://localhost:5146/api/Cloth/GetClothCount
```

Server response

Code

200

Details

Response body

```
7
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Fri,24 Apr 2026 16:43:36 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

TEST 1.2: Get Cloth by ID (Task<Cloth?>)

Request: GET http://localhost:5146/api/Cloth/GetClothById/3

Response (200 OK):

Request URL

```
http://localhost:5146/api/Cloth/GetClothById/3
```

Server response

Code

200

Details

Response body

```
{
  "id": 3,
  "name": "Formal Shirt",
  "category": "Men",
  "type": "Shirt",
  "size": "XL",
  "color": "White",
  "price": 1299,
  "inStock": false
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Fri,24 Apr 2026 16:51:14 GMT
server: Kestrel
transfer-encoding: chunked
```

TEST 1.2: Get Cloth by ID – Not Found (Task<Cloth?>)

Request: GET http://localhost:5146/api/Cloth/GetClothById/99

Response (204 No Content):

Request URL

```
http://localhost:5146/api/Cloth/GetClothById/99
```

Server response

Code | Details

204
Undocumented

Response headers

```
date: Fri,24 Apr 2026 16:55:27 GMT
server: Kestrel
```

ISSUE: Returns 204 instead of 404 – Client cannot differentiate

CATEGORY 2 TESTS: IActionResult (Async – Task<IActionResult>)

TEST 2.1: Get Cloth Details – Success

Request: GET http://localhost:5146/api/Cloth/GetClothDetails/3

Response (200 OK):

Request URL

```
http://localhost:5146/api/Cloth/GetClothDetails/3
```

Server response

Code | Details

200

Response body

```
{
  "id": 3,
  "name": "Formal Shirt",
  "category": "Men",
  "type": "Shirt",
  "size": "XL",
  "color": "White",
  "price": 1299,
  "inStock": false
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Fri,24 Apr 2026 17:02:26 GMT
server: Kestrel
transfer-encoding: chunked
```

TEST 2.2: Get Cloth Details – Not Found

Request: GET http://localhost:5146/api/Cloth/GetClothDetails/99

Response (404 Not Found):

Request URL

http://localhost:5146/api/Cloth/GetClothDetails/99

Server response

Code	Details
404 *Undocumented*	Error: Not Found

Response body

```
Cloth with ID = 99 was not found.
```

Download

Response headers

```
content-type: text/plain; charset=utf-8
date: Fri,24 Apr 2026 17:07:32 GMT
server: Kestrel
transfer-encoding: chunked
```

TEST 2.3: Get Cloths by Category - Success

Request: GET http://localhost:5146/api/Cloth/GetClothsByCategory/men

Response (200 OK):

Request URL

http://localhost:5146/api/Cloth/GetClothsByCategory/men

Server response

Code	Details
200	

Response body


```
[
  {
    "id": 1,
    "name": "Cotton T-Shirt",
    "category": "Men",
    "type": "Shirt",
    "size": "M",
    "color": "Blue",
    "price": 599,
    "inStock": true
  },
  {
    "id": 3,
    "name": "Formal Shirt",
    "category": "Men",
    "type": "Shirt",
    "size": "XL",
    "color": "White",
    "price": 1299,
    "inStock": false
  },
  {
    "id": 4,
    "name": "Sports Shoes",
    "category": "Men",
    "type": "Shoes",
    "size": "9",
    "color": "Grey",
    "price": 3499,
    "inStock": true
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Fri,24 Apr 2026 17:15:56 GMT
server: Kestrel
transfer-encoding: chunked
```

TEST 2.4: Get Cloths by Category – Not Found

Request: GET http://localhost:5146/api/Cloth/GetClothsByCategory/toys

Response (404 Not Found):

Request URL

```
http://localhost:5146/api/Cloth/GetClothsByCategory/toys
```

Server response

Code	Details
404 *Undocumented*	Error: Not Found

Response body

```
No cloths found in category: toys
```

Download

Response headers

```
content-type: text/plain; charset=utf-8
date: Fri,24 Apr 2026 17:22:20 GMT
server: Kestrel
transfer-encoding: chunked
```

CATEGORY 3 TESTS: ActionResult<T> (Async - Task<ActionResult<T>>) - BEST PRACTICE

TEST 3.1: Get Cloth Count Typed (Task<ActionResult<int>>)

Request: GET http://localhost:5146/api/Cloth/GetClothCountTyped

Response (200 OK):

Request URL

```
http://localhost:5146/api/Cloth/GetClothCountTyped
```

Server response

Code	Details
200	

Response body

```
7
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Fri,24 Apr 2026 17:31:34 GMT
server: Kestrel
transfer-encoding: chunked
```

TEST 3.2: Get Cloth by ID Typed (Task<ActionResult<Cloth>>)

Request: GET http://localhost:5146/api/Cloth/GetClothByIdTyped/5

Response (200 OK):

Request URL

http://localhost:5146/api/Cloth/GetClothByIdTyped/5

Server response

Code	Details
200	

Response body


```
{
  "id": 5,
  "name": "Silk Saree",
  "category": "Women",
  "type": "Saree",
  "size": "Free",
  "color": "Red",
  "price": 4999,
  "inStock": true
}
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Fri,24 Apr 2026 17:36:33 GMT
server: Kestrel
transfer-encoding: chunked
```

TEST 3.3: Get Cloth by ID Typed – Not Found

Request: GET http://localhost:5146/api/Cloth/GetClothByIdTyped/99

Response (404 Not Found):

Request URL

http://localhost:5146/api/Cloth/GetClothByIdTyped/99

Server response

Code	Details
404 *Undocumented*	Error: Not Found

Response body

```
Cloth with ID = 99 was not found.
```

Download

Response headers

```
content-type: text/plain; charset=utf-8
date: Fri,24 Apr 2026 17:39:20 GMT
server: Kestrel
transfer-encoding: chunked
```

TEST 3.4: Get by Category Typed (Task<ActionResult<List<Cloth>>>)

Request: GET http://localhost:5146/api/Cloth/GetByCategoryTyped/women

Response (200 OK):

Request URL

```
http://localhost:5146/api/Cloth/GetByCategoryTyped/women
```

Server response

Code	Details
200	

Response body


```
[
  {
    "id": 2,
    "name": "Denim Jacket",
    "category": "Women",
    "type": "Jacket",
    "size": "L",
    "color": "Black",
    "price": 2499,
    "inStock": true
  },
  {
    "id": 5,
    "name": "Silk Saree",
    "category": "Women",
    "type": "Saree",
    "size": "Free",
    "color": "Red",
    "price": 4999,
    "inStock": true
  }
]
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Fri,24 Apr 2026 17:44:49 GMT
server: Kestrel
transfer-encoding: chunked
```

TEST 3.5: Get by Category Typed – Bad Request

Request: GET http://localhost:5146/api/Cloth/GetByCategoryTyped/%20/

Response (400 Bad Request):

Request URL

http://localhost:5146/api/Cloth/GetByCategoryTyped/%20

Server response

Code	Details
400 *Undocumented*	Error: Bad Request

Response body


```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "category": [
      "The category field is required."
    ]
  },
  "traceId": "00-031dbb11d5cad72801397f66ee1400c5-dd07d78844f1fa84-00"
}
```


Download

Response headers

```
content-type: application/problem+json; charset=utf-8
date: Fri,24 Apr 2026 17:48:37 GMT
server: Kestrel
transfer-encoding: chunked
```

localhost:5146/api/Cloth/GetByCategoryTyped/%20/

Import favorites Active nowCSE Ru... YouTube

Pretty-print


```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "category": [
      "The category field is required."
    ]
  },
  "traceId": "00-a3e75a34fa9764c03058941306cc8664-a6f6f67cdf4467e7-00"
}
```


N.B.: in “http://localhost:5146/api/Cloth/GetByCategoryTyped/%20/” , “%20/” represents the empty string, and it can be written on the address bar as ” http://localhost:5146/api/Cloth/GetByCategoryTyped/ <A free space> /” before hit, and “%20/” automatically added.

Best Practice Recommendation

Use **ActionResult<T>** with async for all production APIs.

```
// BEST PRACTICE - Async ActionResult<T>
[HttpGet("{id}")]
public async Task<ActionResult<Cloth>> GetClothById(int id)
{
    if (id <= 0)
        return BadRequest("Invalid ID");

    var cloth = await ClothService.GetClothByIdAsync(id);

    if (cloth == null)
        return NotFound($"Cloth with ID {id} not found");

    return Ok(cloth);
}
```

Why?

- Async – Scalable, non-blocking
- Strongly typed – Swagger shows extra schema
- Full HTTP control – 200, 400, 404, 201, etc.
- Compile-time type checking
- Self-documenting API
- Client-friendly (predictable responses)